Michael Prieto

El Paso, TX 79936 | (915) 800-3423 | michaelprieto0930@gmail.com | linkedin.com/in/michaelfprieto

SUMMARY

Results-driven Senior Full Stack Engineer with 8+ years of experience designing and developing scalable, high-performance web and cloud-based applications across healthcare, fintech, and retail industries. Expert in Java, Spring Boot, React, Angular, and .NET, with a proven track record of modernizing legacy systems, architecting microservices, and integrating machine learning models using FastAPI and gRPC. Adept at building secure, responsive frontend interfaces and RESTful/gRPC APIs, while leveraging AWS, Azure, Docker, and Kubernetes for cloud-native deployment and DevOps automation. Strong advocate for Test-Driven Development (TDD), Domain-Driven Design (DDD), and Agile methodologies, with hands-on experience in building event-driven systems using Apache Kafka, and managing both relational and NoSQL databases like PostgreSQL, MySQL, SQL Server, MongoDB, and Redis. Passionate about clean code, cross-functional collaboration, and delivering robust software solutions that drive business value and user satisfaction.

EDUCATION

Bachelor of Science in Computer Engineering | Texas State University | San Marcos, TX
Degree obtained May 2016

EXPERIENCE

Sr. Java Full Stack Engineer | IDEXX | Westbrook, ME
Mar 2024 – Mar 2025

- Architected and engineered high-performance veterinary laboratory applications using **Java**, **Spring Boot** for the VetLab Station project, achieving a 30% improvement in application performance and enhancing the efficiency of lab data processing workflows.
- Refactored legacy monolithic applications by applying **Domain-Driven Design (DDD)** principles and introducing a **microservices**-based architecture, significantly improving system scalability, modularity, and maintainability.
- Built secure and scalable **RESTful APIs** and **gRPC** services with **Spring Boot** and **Spring Cloud** to facilitate low-latency communication between microservices and **Python**-based ML inference engines.
- Integrated **Python**-based machine learning models into the diagnostic platform using **FastAPI** and **gRPC**, enabling real-time predictions for specimen analysis and improving diagnostic accuracy by 25%.
- Developed **Python** scripts to preprocess and validate clinical data before feeding it into ML models, ensuring data integrity and consistency across the pipeline.
- Designed and developed modular **React** components for displaying real-time diagnostic results, specimen statuses, and lab workflow stages. Components included result tables, trend graphs, sample detail modals, and pagination-enabled search lists.
- Utilized **React Hooks** to manage state across dynamic form inputs, result refresh intervals, and authenticated user sessions without relying on class components.
- Implemented **Redux** for global state management, ensuring consistent handling of sample data across various views such as result dashboards, lab queues, and patient history pages.
- Streamlined lab workflows by integrating third-party APIs and internal lab equipment systems, reducing development timelines by 32%.
- Designed and optimized **SQL Server** schemas, stored procedures, and complex queries to support large-scale lab operations including diagnostic retrieval, trend analysis and sample lifecycle tracking.

- Applied **Spring Data JPA** and **Hibernate ORM** for seamless entity mapping, criteria-based querying, and transaction management, reducing boilerplate code and enhancing developer productivity.
- Implemented real-time data streaming and processing using **Apache Kafka**, reducing data latency by 70% and improving the timelines of critical lab alerts, ensuring prompt decision-making in veterinary diagnostics.
- Deployed **Java microservices** and **Python** ML inference APIs on **AWS** using **ECS** for container orchestration, **RDS** for managed database hosting, and **S3** for secure diagnostic document storage.
- Containerized services with **Docker** and implemented automated **CI/CD** pipelines with **AWS CodePipeline** and **GitHub Actions**, ensuring rapid and reliable delivery of new features and updates.
- Implemented automated unit and integration tests using **JUnit**, **Mockito**, and **PyTest** to achieve high test coverage and ensure reliability across **Java**, **gRPC**, and **Python**-based services.
- Used **Cypress** for end-to-end testing of **React**-based workflows, ensuring application reliability and minimizing regressions during frequent deployments.
- Collaborated with cross-functional **Agile** teams including DevOps, QA, data science, and veterinary domain experts, using **Kanban** to prioritize tasks and ensure continuous, regulatory-compliant delivery.
- Led code reviews, mentored junior engineers, and contributed to continuous improvement initiatives across the full software development lifecycle.

Full Stack Engineer/Senior Full Stack Engineer | IWP Capital | Fort Worth, TX
Nov 2019 – Feb 2024

- Designed, developed, and deployed several innovative fintech solutions using **Java, Spring Boot, Angular and React**, ensuring seamless integration of backend services with high-performance, responsive end-user UIs. Led key architectural decisions that significantly improved system efficiency and user experience.
- Identified and defined the core business functionalities of the application that could be divided into independent **microservices**, ensuring each service had a single responsibility. Created the user management service, investment service, portfolio service, and notification service.
- Designed the backend architecture by implementing **Spring Boot microservices**, ensuring each service was loosely coupled, independently deployable, and able to interact with other services over **REST API**s.
- Implemented **Test-Driven Development (TDD)** across multiple microservices, including User Management, Investment, and Portfolio Services, ensuring that all business logic was thoroughly tested before implementation.
- Developed and exposed **RESTful API**s for each microservice using **Spring Boot**.
- Integrated **JWT**-based authentication across all microservices, ensuring secure and tokenized user sessions that were consistent across different services.
- Configured **Spring Security** to implement role-based access control, allowing users to access specific resources based on their roles and permissions.
- Led the frontend development of the Sanctify App, a faith-based fintech platform, for both web and mobile users. Built a responsive, dynamic UI with **React** for web, ensuring a seamless experience across devices. Implemented **React Router** for efficient navigation and optimized load times using lazy loading.
- Developed the mobile app with **React Native**, providing cross-platform support for iOS and Android. Ensured the app's performance by reducing load time by 25% through code splitting and caching strategies.
- Designed a clean, intuitive responsive UI for the Proxy Voting Platform using **Angular Material** and integrated backend services using **ASP.NET microservices**.
- Built secure **RESTful API**s with **ASP.NET Web API** for handling voting proposals, meeting schedules, and user submissions.
- Built an interactive data visualization dashboard using **D3.js**, allowing users to view trends in their investment data, including portfolio performance, risk metrics, and historical performance, with dynamic, real-time updates.
- Implemented real-time updates on voting activity using **SignalR**, improving responsiveness and user engagement.
- Ensured each microservice had its own database using **PostgreSQL** and **MongoDB**, providing complete decoupling between services and reducing dependencies. Used **Spring Data JPA** for efficient database interaction, utilizing **PostgreSQL** for relational data (user information, etc.) and **MongoDB** for unstructured data (user preferences, logs, etc.).

- Used **Redis** for caching frequently accessed data, ensuring faster response times for users accessing financial data or portfolio updates.
- Implemented **Spring Cloud Netflix Eureka** for service discovery, allowing services to register and discover each other dynamically without hardcoding URLs.
- Used **Ribbon** for client-side load balancing, ensuring that requests were distributed evenly across instances of microservices, improving performance and resilience.
- Integrated **Apache Kafka** to handle asynchronous messaging and data synchronization between microservices, allowing real-time processing of data updates. Enabled communication between the Investment Service and the User Service to share real-time investment data and ensure users had up-to-date portfolio information.
- Deployed the Sanctify App on **AWS**, leveraging **Amazon EC2** instances for hosting backend services and **AWS Elastic Beanstalk** for simplified deployment and scaling of the application.
- Integrated **Amazon RDS** to store and manage financial data, using **PostgreSQL** for database management, and **AWS S3** for securely storing user documents, transaction logs, and other unstructured data.
- Implemented **AWS Lambda** functions for serverless processing of background tasks such as generating reports and sending notifications, reducing infrastructure overhead.
- Used **AWS CloudWatch** for monitoring and logging, enabling proactive issue detection and real-time performance insights, ensuring high availability and fast response times across the application.
- Deployed **.NET microservices** for the Proxy Voting Platform on **Azure Kubernetes Service (AKS)** using **Docker** containers, enabling scalable and resilient cloud-native infrastructure.
- Used **Azure SQL Database** and **Entity Framework Core** for data persistence, enabling scalable and secure access to structured voting records and user information.
- Integrated **Azure Blob Storage** for storing large files like voting records and corporate documents. This ensured secure, scalable, and cost-efficient storage.
- Established and maintained **CI/CD pipelines** using **Jenkins**, **GitLab**, and **SonarQube**, automating build, testing, and deployment workflows. Integrated static code analysis and automated testing frameworks like **JUnit**, and **Selenium**, significantly reducing deployment times by 50% and ensuring high code quality and security.
- Used **Docker** to containerize each microservice, ensuring consistency across development, testing, and production environments. **Kubernetes** was used for orchestrating the deployment of services in a cloud environment.
- Used **Cucumber** to run the **Behavior-Driven Development (BDD)** tests as part of the continuous integration pipeline in **GitLab CI**, ensuring that all user stories were tested before deployment.
- Actively collaborated with business stakeholders and cross-functional teams in an **Agile Scrum environment**, participating in sprint planning, daily stand-ups, code reviews, and sprint retrospectives to ensure alignment and continuous improvement.

Full Stack Developer | The Home Depot | Atlanta, GA
Sep 2016 – Octo 2019

- Implemented a **microservices**-based inventory management system using **Java** technologies, ensuring seamless integration into the scalable e-commerce platform.
- Leveraged **Spring Boot** and **Spring MVC** to develop a robust backend of ecommerce platform, implementing core functionalities such as user authentication, product catalog management, and order processing, which provided a seamless user experience.
- Integrated security measures such as **Spring Security**, **OAuth2**, and **JWT** to protect sensitive data, ensuring compliance with industry standards and enhancing customer trust.
- Developed a standalone product catalog module using **Angular** within a micro-frontend architecture, ensuring seamless integration with other platform components and enhancing the modularity of the e-commerce system.
- Collaborated closely with **UX/UI** designers to craft an intuitive DIY project builder tool, personally contributing to the design and user flow, enabling users to seamlessly select materials, tools, and instructions tailored to their specific needs, which led to a 31% increase in user engagement and satisfaction.
- Created **REST API**s for real-time inventory tracking, integrating with existing systems and third-party services, which allowed for accurate inventory updates and improved customer experience.

- Utilized Google AI's **TensorFlow** to build predictive models that forecasted demand for specific products, allowing for better inventory planning and reducing stockouts or overstock situations.
- Integrated **Google BigQuery** into the e-commerce platform to handle large-scale data analytics, enabling real-time processing and analysis of vast datasets, such as customer behavior, transaction history, and inventory trends.
- Utilized **MySQL** as part of the hybrid relational data strategy, optimizing transactional operations such as product management, order history, and customer account data alongside **AWS RDS** for broader scalability.
- Configured and optimized **MySQL** indexes, queries, and schema design to ensure efficient performance for high-volume read/write operations in the core inventory and checkout systems.
- Implemented **Amazon DynamoDB** for managing non-relational data, such as session logs and customer interaction data, ensuring low-latency access and high availability.
- Used **AWS CloudFront** and **S3** to serve DIY project images and assets, optimizing load times and ensuring a fast, responsive user experience across different devices.
- Deployed scalable microservices on **Amazon EC2** and orchestrated them using **Amazon EKS**, ensuring efficient resource utilization and seamless application scaling.
- Implemented an event-driven architecture using **Apache Kafka** to enable seamless communication between services and used the Saga pattern to manage distributed transactions effectively.
- Implemented **RabbitMQ** to handle task-based messaging, such as triggering rental confirmations, managing equipment availability, and updating inventory in real-time.
- Integrated **Test-Driven Development (TDD)** with continuous integration tools such as **Jenkins**, ensuring that code was automatically tested with every commit.
- Adopted **CI/CD pipelines** for continuous integration and deployment, using **Jenkins** and **Terraform** to provision **infrastructure as code (IaC)**, ensuring that updates were delivered quickly and reliably without disrupting the overall system.
- Utilizing **Selenium** for end-to-end testing improved the consistency of the user experience across the installation service, DIY implementation, and inventory integration by 10%.
- Managed version control and collaborative development using **GitHub**, implementing branching strategies, handling pull requests, and resolving merge conflicts to ensure smooth collaboration across frontend and backend teams, supporting an efficient development workflow.
- Collaborated within a large-scale **Agile** team using **SAFe framework** to manage and deliver high-quality software products. Worked closely with product owners, scrum masters and other stakeholders to refine requirements and ensure successful feature delivery.

SKILLS

Programming Languages	Java, C#, Python, JavaScript, TypeScript
Backend Frameworks	Spring Boot, Spring MVC, Spring Cloud, Hibernate, JPA, ASP.NET, ASP.NET Web API, Entity Framework, FastAPI
Frontend Frameworks	React, React Native, Angular
API Integration	RESTful API, gRPC, SignalR, WebSockets, OAuth2, JWT
DevOps & CI/CD	Docker, Kubernetes (EKS, AKS), Jenkins, GitHub Actions, GitLab CI/CD, AWS CodePipeline, Azure DevOps, SonarQube, Terraform
Messaging Systems	Apache Kafka, RabbitMQ
Databases	PostgreSQL, SQL Server, MySQL, MongoDB, Amazon DynamoDB, Redis, Google BigQuery
Cloud Platforms	Amazon Web Services (EC2, ECS, EKS, RDS, S3, Lambda, CloudFront, CloudWatch) Microsoft Azure (AKS, Azure SQL, Blob Storage)
Testing & QA	JUnit, Mockito, PyTest, Selenium, Cypress, Cucumber